Neural Intelligence

Deep Learning: Foundations and Probabilistic Perspective

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
Homepage

March 2026

What is neural intelligence?

Neural intelligence refers to computation and cognition arising from networks of simple, interconnected processing units — inspired by the nervous system.

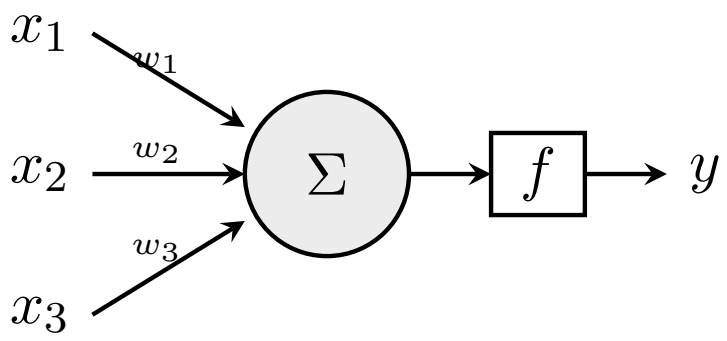
Main idea: *intelligence emerges from structure and learning*, not explicit programming.

What is neural intelligence?

- Biological brains: $\sim 86 \times 10^9$ neurons, each connected to $\sim 10^4$ others
- Artificial neural networks (ANNs): parametric function approximators trained from data
- Deep learning: ANNs with many (at least 1 hidden) layers, learning hierarchical representations

Biological neuron

- Dendrites receive weighted input signals
- Cell body integrates (sums) inputs
- If integrated potential exceeds a threshold, the axon fires an action potential
- Synaptic weights are modifiable (Hebbian learning)



The abstract model retains:

weighted inputs + threshold + nonlinear activation.

Reference: McCulloch & Pitts, *Bull. Math. Biophysics*, 1943.

The artificial neuron

Model (McCulloch–Pitts, 1943):

$$y = f\left(\sum_{j=1}^{d} w_j x_j + b\right) = f\Big(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b\Big)$$

where $\boldsymbol{x} \in \mathbb{R}^d$: input, $\boldsymbol{w} \in \mathbb{R}^d$: weight vector, $b \in \mathbb{R}$: bias, f: nonlinear activation.

The artificial neuron

Observation

A single neuron computes a *linear combination* of inputs followed by a *nonlinearity*. The power of deep networks comes from composing many such units.

Terminology: unit, node, perceptron, neuron — used interchangeably in the literature.

Perceptron: linear classifier

Rosenblatt (1958):

$$y = \mathrm{sign}(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b), \quad y \in \{-1, +1\}$$

Perceptron update rule (online, for misclassified sample $(\boldsymbol{x}^{(i)}, t^{(i)})$):

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\, t^{(i)}\boldsymbol{x}^{(i)}, \quad b \leftarrow b + \eta\, t^{(i)}$$

Perceptron: linear classifier

Theorem (Perceptron Convergence)

If training data is linearly separable, the perceptron algorithm converges in a finite number of steps.

Limitation: Minsky & Papert (1969) showed a single perceptron cannot solve XOR. $\Rightarrow$ motivates multi-layer networks.

Reference: Rosenblatt, *Psych. Rev.*, 1958; Minsky & Papert, *Perceptrons*, MIT Press, 1969.

Multi-layer perceptron (MLP)

Arrange neurons in layers:

$$\underbrace{\boldsymbol{x}}_{\text{input}} \longrightarrow \underbrace{\mathbf{h}^{(1)}, \ldots, \mathbf{h}^{(L-1)}}_{\text{hidden layers}} \longrightarrow \underbrace{\hat{\mathbf{y}}}_{\text{output}}$$

For layer ℓ:

$$\mathbf{h}^{(\ell)} = f^{(\ell)}\left(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad \mathbf{h}^{(0)} = \boldsymbol{x}$$

The network computes a composition of nonlinear maps:

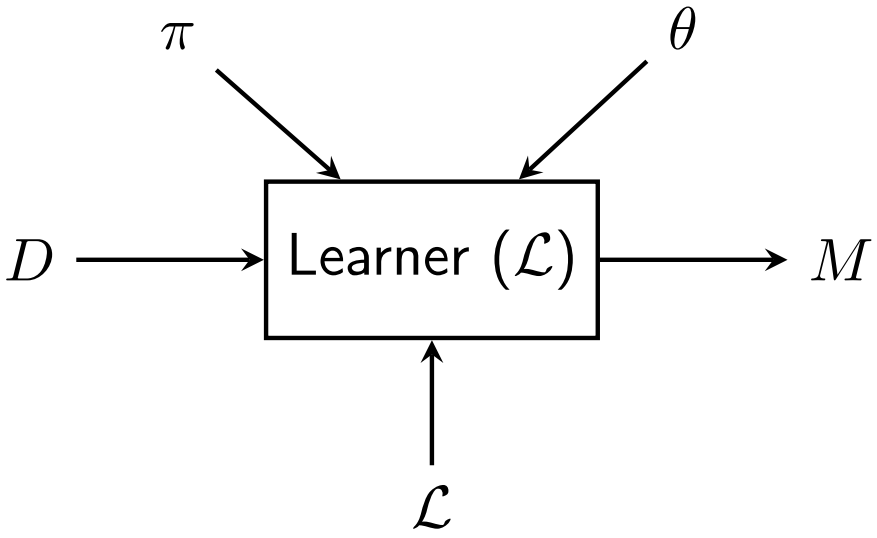
$$F(\boldsymbol{x};\theta) = f^{(L)}\Big(W^{(L)} \cdots f^{(1)}\big(W^{(1)}\boldsymbol{x} + \mathbf{b}^{(1)}\big) + \cdots\Big)$$

where $\theta = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$ are learnable parameters.

Components of deep learning

A deep learning system is characterised by five components:

- D: Dataset
- M: Model
- π: Structure of the model
- θ: Parameters of the model
- $\mathcal{L}$: A loss function



Reference: T. Dash et al., arXiv:2107.10295, 2021.

Components: definitions

D – Dataset: A set of labelled examples $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$; y is a class label or a real value.

M – Model: Mathematical machinery for ingesting $\boldsymbol{x}$ and computing predictions via successive transformations ("deep net").

π – Structure: Architecture: number of layers, neurons per layer, and their interconnection pattern.

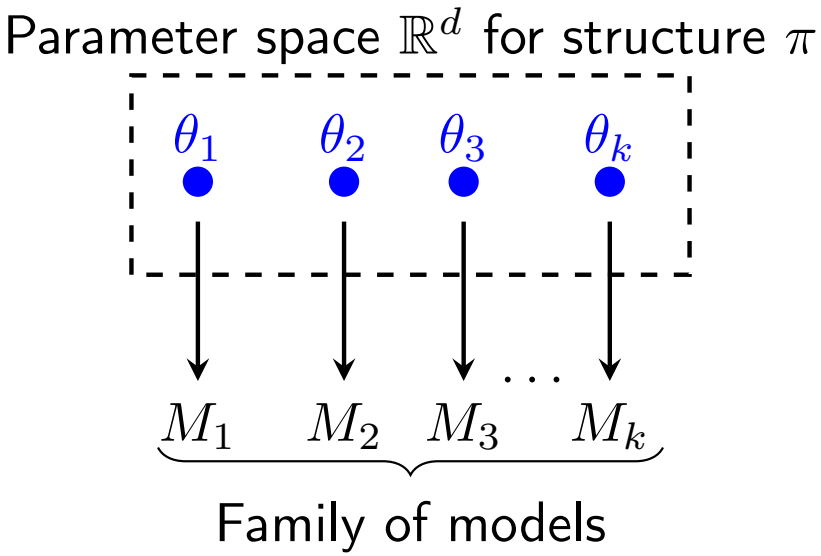
θ – Parameters: Connection strengths (synaptic weights); each connection carries a real-valued weight.

$\mathcal{L}$ – Loss: A formal measure of prediction error. We always aim to *minimise* $\mathcal{L}$.

The *learner* $\mathcal{L}$ uses D to find the best parameter set θ^* for model M with structure π by minimising $\mathcal{L}$.

Learning as search over a model family

Parameters θ are directly determined by structure π. Varying θ within a fixed π produces a *family of models*:



Learning as search over a model family

- There can be *infinitely many* model instances in a family.
- It is infeasible to enumerate all models; we need a tractable search procedure.
- Gradient descent provides such a search: moving through $\mathbb{R}^d$ guided by $\nabla_\theta \mathcal{L}$.

Universal approximation

Theorem (Cybenko, 1989; Hornik et al., 1989)

A feedforward network with a single hidden layer of sufficient width and a non-polynomial activation can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision.

- This is an *existence* result — it does not say how to find the network.
- Width may need to be exponential in input dimension.
- Depth is more parameter-efficient: some functions require exponentially fewer parameters when expressed as deep networks (Montufar et al., NeurIPS 2014).

Practical takeaway: depth $>$ width for representation efficiency. This is the core motivation for *deep* learning.

Reference: Cybenko, *Math. Control Sig. Sys.*, 1989; Hornik et al., *Neural Networks*, 1989.

Deep learning from a Bayesian perspective

What does the search over the model family really mean probabilistically?

A model is defined by its structure π and its parameters $\boldsymbol{w}$ (we use $\boldsymbol{w}$ for weights; synonymous with θ).

A search over model instances, given structure π, is a search over the *parameter space* $\mathbb{R}^d$. We represent *prior beliefs* about what the parameters are via a prior distribution:

$$p(\boldsymbol{w})$$

Deep learning from a Bayesian perspective

As we collect data $D = (\mathbf{X}, \mathbf{Y})$, we update this belief to a *posterior*:

$$p(\boldsymbol{w} \mid D) = \frac{p(D \mid \boldsymbol{w})\, p(\boldsymbol{w})}{p(D)}$$

Reference: Bishop, *PRML*, Cambridge, 2006, Ch. 5.

The posterior distribution

Expanding with $D = (\mathbf{X}, \mathbf{Y})$:

$$p(\boldsymbol{w} \mid \mathbf{X}, \mathbf{Y}) = \frac{p(\mathbf{Y} \mid \mathbf{X}, \boldsymbol{w})\, p(\boldsymbol{w})}{p(\mathbf{Y} \mid \mathbf{X})}$$

- $p(\mathbf{Y} \mid \mathbf{X}, \boldsymbol{w})$: *likelihood* — how probable is the data given $\boldsymbol{w}$?
- $p(\boldsymbol{w})$: *prior* — our belief about $\boldsymbol{w}$ before seeing data.
- $p(\mathbf{Y} \mid \mathbf{X})$: *marginal likelihood* (evidence) — a normalisation constant.

The posterior distribution

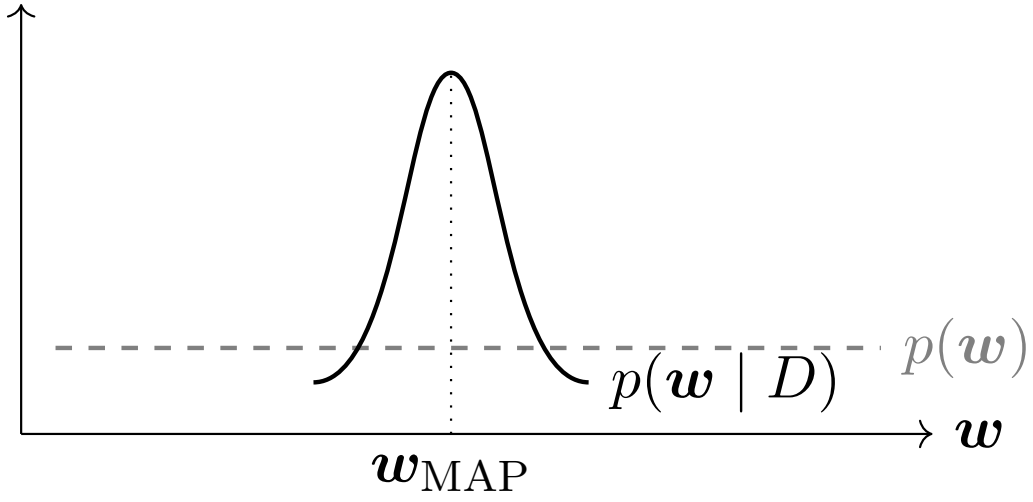
Since $p(\mathbf{Y} \mid \mathbf{X}) = \int p(\mathbf{Y} \mid \mathbf{X}, \boldsymbol{w})\, p(\boldsymbol{w})\, d\boldsymbol{w}$ is independent of $\boldsymbol{w}$:

$$\boxed{p(\boldsymbol{w} \mid D) \;\propto\; p(D \mid \boldsymbol{w})\, p(\boldsymbol{w})}$$

Interpretation: searching for the best model = finding the $\boldsymbol{w}$ that maximises this posterior. The denominator (evidence) does not affect the search.

Prior to posterior: belief update

Learning the weights means shifting belief from $p(\boldsymbol{w})$ to $p(\boldsymbol{w} \mid D)$:



A broad (uncertain) prior $p(\boldsymbol{w})$ becomes a peaked posterior $p(\boldsymbol{w} \mid D)$ once we observe data.

The mode of the posterior is $\boldsymbol{w}^{\mathrm{MAP}}$.

Maximum likelihood estimation (MLE)

A neural network represents a function $f(\boldsymbol{x}, \boldsymbol{w})$. Its output $\omega = f(\boldsymbol{x}, \boldsymbol{w})$ *parameterises a distribution* over y; it is *not* a direct prediction of y.

The network's goal is to estimate the conditional likelihood $p(\mathbf{Y} \mid \mathbf{X}, \boldsymbol{w})$.

Maximum likelihood estimation (MLE)

MLE: find $\boldsymbol{w}$ that maximises the log-likelihood:

$$\boldsymbol{w}^{\mathrm{MLE}} = \arg\max_{\boldsymbol{w}} \log p(D \mid \boldsymbol{w}) = \arg\max_{\boldsymbol{w}} \sum_i \log p(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{w})$$

Equivalently, minimise the *negative log-likelihood* (NLL):

$$\mathcal{L}_{\mathrm{NLL}}(\boldsymbol{w}) = -\sum_i \log p(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{w})$$

Key point: different distributional assumptions on y yield different standard loss functions.

Reference: Goodfellow et al., *Deep Learning*, Ch. 6.

MLE $\Rightarrow$ MSE for regression

Assume $y^{(i)} = f(\boldsymbol{x}^{(i)}, \boldsymbol{w}) + \epsilon^{(i)}$ with $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$.

$$p(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{w}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\left(y^{(i)} - f(\boldsymbol{x}^{(i)}, \boldsymbol{w})\right)^2}{2\sigma^2} \right)$$

Log-likelihood:

$$\log \mathcal{L}(\boldsymbol{w}) = -\frac{N}{2} \log(2\pi\sigma^2) \; - \; \frac{1}{2\sigma^2} \sum_{i=1}^{N} \left(y^{(i)} - f(\boldsymbol{x}^{(i)}, \boldsymbol{w})\right)^2$$

Maximising $\log \mathcal{L} \equiv$ minimising:

$$\boxed{\mathcal{L}_{\mathrm{MSE}}(\boldsymbol{w}) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - f(\boldsymbol{x}^{(i)}, \boldsymbol{w})\right)^2}$$

MSE is NLL under a Gaussian output assumption.

MLE $\Rightarrow$ cross-entropy for classification

For K-class classification, assume $p(y = k \mid \boldsymbol{x}, \boldsymbol{w}) = \hat{p}_k$ where:

$$\hat{\mathbf{p}} = \mathrm{softmax}(f(\boldsymbol{x}, \boldsymbol{w})), \quad \hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

NLL with one-hot target $\mathbf{t}^{(i)}$:

$$-\log p(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{w}) = -\sum_{k=1}^{K} t_k^{(i)} \log \hat{p}_k^{(i)}$$

Summing over data:

$$\boxed{\mathcal{L}_{\mathrm{CE}}(\boldsymbol{w}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} t_k^{(i)} \log \hat{p}_k^{(i)}}$$

Cross-entropy loss is NLL under a categorical output assumption.

Maximum a posteriori (MAP) and regularisation

Incorporating a prior $p(\boldsymbol{w})$ via Bayes' rule:

$$\begin{aligned}\boldsymbol{w}^{\mathrm{MAP}} &= \arg\max_{\boldsymbol{w}} \log p(\boldsymbol{w} \mid D) \\ &= \arg\max_{\boldsymbol{w}} \left[\sum_i \log p(y^{(i)} \mid \boldsymbol{x}^{(i)}, \boldsymbol{w}) + \log p(\boldsymbol{w}) \right]\end{aligned}$$

Maximum a posteriori (MAP) and regularisation

The term $\log p(\boldsymbol{w})$ acts as a *regulariser*:

Prior $p(\boldsymbol{w})$	$\log p(\boldsymbol{w})$	Regulariser
$\mathcal{N}(0, \tau^2 I)$	$-\frac{1}{2\tau^2}\|\boldsymbol{w}\|_2^2 + \text{const}$	L_2 (weight decay)
Laplace$(0, b)$	$-\frac{1}{b}\|\boldsymbol{w}\|_1 + \text{const}$	L_1 (sparsity)

Weight decay $\lambda\|\boldsymbol{w}\|_2^2$ (most common in practice) is *exactly* MAP with a Gaussian prior.

Reference: Bishop, *PRML*, Ch. 3.3; Goodfellow et al., Ch. 7.

Network as a distribution parameteriser

The general picture:

(1) The network $f(\boldsymbol{x}, \boldsymbol{w})$ outputs $\omega = f(\boldsymbol{x}, \boldsymbol{w})$: parameters of a distribution over y.

(2) For regression: $\omega = \mu$ (mean of Gaussian).

(3) For classification: $\omega = \mathbf{p}$ (probabilities via softmax).

(4) The loss function is the NLL of this distribution:

$$\mathcal{L}(\boldsymbol{w}) = -\log p(y \mid \omega(\boldsymbol{x}))$$

Unified view

MLE/MAP principles provide a *principled guide* to designing loss functions for any output type: regression, classification, density estimation, generation, etc.

Reference: Goodfellow et al., *Deep Learning*, Ch. 6.

Backpropagation

Goal: compute $\nabla_\theta \mathcal{L}(\theta)$ efficiently (reverse-mode automatic differentiation). Define pre-activation $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ and error signal:

$$\boldsymbol{\delta}^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$$

Recursive backpropagation (chain rule):

$$\boldsymbol{\delta}^{(\ell)} = \left(W^{(\ell+1)^\top} \boldsymbol{\delta}^{(\ell+1)} \right) \odot f'\left(\mathbf{z}^{(\ell)}\right)$$

Weight gradient:

$$\nabla_{W^{(\ell)}} \mathcal{L} = \boldsymbol{\delta}^{(\ell)} \mathbf{h}^{(\ell-1)^\top}$$

Backpropagation is *not* a learning algorithm; it is a gradient computation method. The learning algorithm (SGD, Adam) uses these gradients.

Reference: Rumelhart, Hinton & Williams, *Nature*, 1986; AIMA 4th ed., §21.2.

Gradient descent and Adam

Mini-batch SGD parameter update:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}_\mathcal{B}(\theta)$$

Adam (Kingma & Ba, ICLR 2015) — the standard adaptive optimiser:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

where m_t, v_t are exponential moving averages of gradient and squared gradient.

- Combines momentum (first moment $\hat{m}_t$) with per-parameter adaptive rates (second moment $\hat{v}_t$).
- Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Reference: Kingma & Ba, *ICLR*, 2015.

Activation functions

Name	Formula	Notes
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	saturates; vanishing gradient
Tanh	$\tanh(z)$	zero-centred; still saturates
ReLU	$\max(0, z)$	sparse, fast; dying ReLU issue
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \ll 1$	fixes dying ReLU
GELU	$z\,\Phi(z)$	smooth; default in Transformers
Swish	$z \cdot \sigma(\beta z)$	self-gated, non-monotone

Activation functions

Why non-linearity?

Without f, a composition of linear maps is itself linear: $W^{(L)} \cdots W^{(1)} \boldsymbol{x} = W \boldsymbol{x}$. Nonlinearities enable hierarchical feature construction.

Reference: Nair & Hinton, *ICML*, 2010 (ReLU); Hendrycks & Gimpel, 2016 (GELU).

Vanishing and exploding gradients

In a network of depth L, the gradient w.r.t. layer-1 weights involves a product of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} \propto \prod_{\ell=2}^{L} W^{(\ell)} \cdot \mathrm{diag}\Big(f'\Big(\mathbf{z}^{(\ell)}\Big)\Big)$$

- $\|W^{(\ell)}\| < 1$ or saturating $f' \Rightarrow$ vanishing gradients; early layers learn slowly.
- $\|W^{(\ell)}\| > 1 \Rightarrow$ exploding gradients; training diverges.

Vanishing and exploding gradients

Mitigations:

- ReLU/GELU activations (non-saturating for $z > 0$)
- Residual (skip) connections: He et al., *CVPR*, 2016
- Batch normalisation: Ioffe & Szegedy, *ICML*, 2015
- Xavier/He initialisation

Regularisation

(1) **Weight decay** (L_2 prior / MAP): $\mathcal{L} \leftarrow \mathcal{L} + \lambda \|\theta\|_2^2$

(2) **Dropout** (Srivastava et al., JMLR 2014): randomly zero out units during training with probability p; approximates an ensemble of 2^H sub-networks

(3) **Batch normalisation**: normalise pre-activations; reduces internal covariate shift and regularises

(4) **Data augmentation**: expand training set via label-preserving transforms

(5) **Early stopping**: halt when validation loss starts increasing

Regularisation

Bias–variance interpretation

Regularisation reduces variance (overfitting) at the cost of a small increase in bias.

Reference: Srivastava et al., *JMLR*, 2014; Ioffe & Szegedy, *ICML*, 2015.

Modern deep architectures

Architecture	Inductive bias	Typical use
CNN	local connectivity, weight sharing	images, 1D sequences
RNN/LSTM	sequential order, recurrent state	time series, NLP
Transformer	global attention, positional encoding	NLP, vision, biology
GNN	graph structure	molecules, knowledge graphs
VAE	latent variable, ELBO	generative modelling
GAN	adversarial training	image synthesis

Modern deep architectures

Each architecture encodes a different *structural prior* about the data. Choosing the right architecture = choosing the right inductive bias.

Reference: AIMA 4th ed., Ch. 21; Goodfellow et al., Chs. 9–10; Vaswani et al., *NeurIPS*, 2017.

Deep learning in AI: the big picture

Where does deep learning sit?

$$\underbrace{\text{AI}}_{\text{rational agents}} \supset \underbrace{\text{ML}}_{\text{learn from data}} \supset \underbrace{\text{DL}}_{\text{hierarchical feature learning}}$$

Deep learning in AI: the big picture

Strengths of DL:

- Automatic feature engineering from raw data
- Scales with data and compute (empirical scaling laws)
- State-of-the-art in perception, language, generation

Deep learning in AI: the big picture

Limitations (motivating neuro-symbolic AI):

- Requires large labelled datasets; poor sample efficiency
- Opaque representations; limited interpretability
- Struggles with systematic generalisation and formal reasoning

Reference: Marcus, *arXiv:1801.00631*, 2018; LeCun, Bengio & Hinton, *Nature*, 2015.

Summary

(1) A deep learning system has five components: D, M, π, θ, $\mathcal{L}$.

(2) Learning = search over a family of models; GD makes this tractable.

(3) Deep networks are provably more expressive per parameter than shallow ones.

(4) Loss functions follow directly from MLE under assumed output distributions (Gaussian $\Rightarrow$ MSE; categorical $\Rightarrow$ CE).

(5) Regularisation corresponds to MAP with a prior on weights ($\mathcal{N} \Rightarrow L_2$; Laplace $\Rightarrow L_1$).

(6) The network output $f(\boldsymbol{x}, \boldsymbol{w}) = \omega$ parameterises a distribution; the loss is $-\log p(y \mid \omega(\boldsymbol{x}))$.

(7) Backpropagation computes gradients.

(8) Architecture choice encodes inductive bias.

References

(1) S. Russell & P. Norvig. *AIMA*, 4th ed., 2020. Ch. 21.

(2) I. Goodfellow, Y. Bengio & A. Courville. *Deep Learning*, MIT Press, 2016.

(3) C. Bishop. *Pattern Recognition and Machine Learning*, Springer, 2006.

(4) T. Dash et al. A review and refinement of macro-level taxonomies of types of inductive biases. arXiv:2107.10295, 2021.

(5) Y. LeCun, Y. Bengio & G. Hinton. Deep learning. *Nature*, 521:436–444, 2015.

(6) D. Rumelhart, G. Hinton & R. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

(7) G. Cybenko. Approximation by superpositions of a sigmoidal function. *Math. Control Sig. Sys.*, 2:303–314, 1989.

(8) D. Kingma & J. Ba. Adam: a method for stochastic optimisation. *ICLR*, 2015.

(9) N. Srivastava et al. Dropout. *JMLR*, 15:1929–1958, 2014.

(10) K. He et al. Deep residual learning for image recognition. *CVPR*, 2016.

(11) A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.